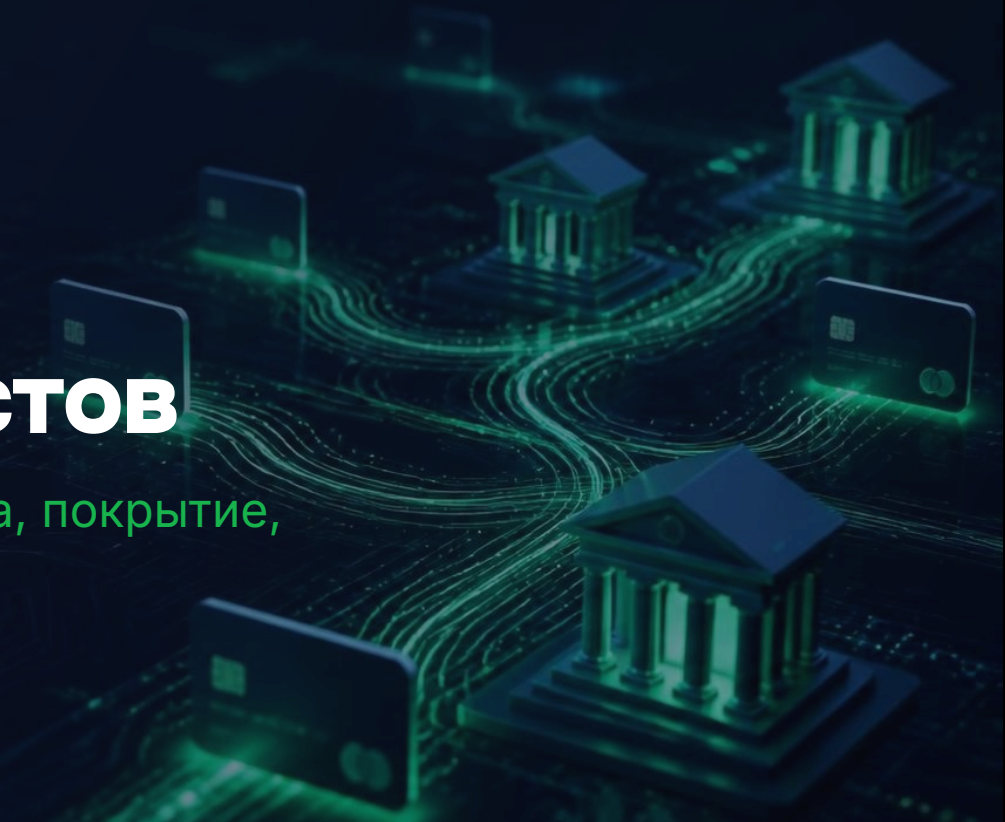
ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Качество unit-тестов

Модуль 4 · Лекция 8 · Четыре столпа, покрытие, maintainability

Ирина Габанова · Lekton

4 курс



МОДУЛЬ 4 · ЛЕКЦИЯ 8

План лекции

01 Проблема: тесты есть, а уверенности нет

02 Четыре столпа хорошего теста (Хориков)

03 Метрики: Branch Coverage, JaCoCo, закон Гудхарта

04 Мутационное тестирование (PITest)

05 Интерактив: параметризованные тесты — выбери источник данных

06 Читаемость, хрупкость, overmocking, стили тестирования

07 Интерактив: разбор плохих тестов + чек-лист ревьюера

МОДУЛЬ 4 · ЛЕКЦИЯ 8

Проблема: тесты есть, а уверенности нет

Приборная панель: всё зелёное

800 unit-тестов, JaCoCo показывает 85% покрытия, все индикаторы зелёные.

…и инцидент в проде

После релиза клиентам неверно считает комиссию. Метод с багом «покрыт тестами» — но ни один не проверяет результат расчёта.

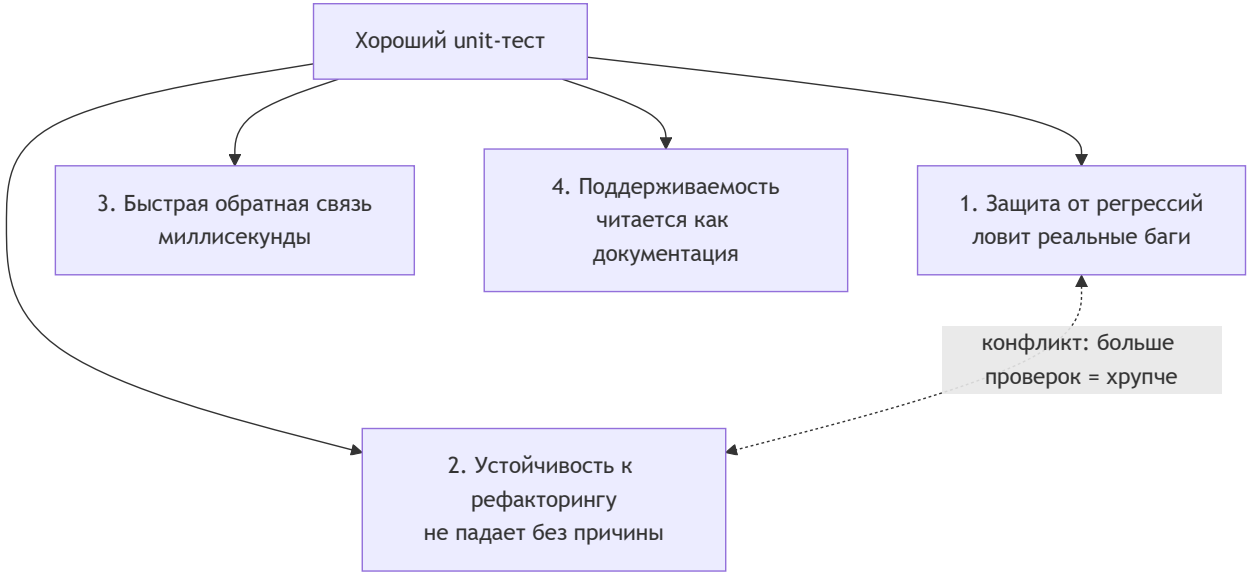
Тезис лекции: тесты зелёные ≠ код работает. Тест может выполнять код и не проверять ничего.

Признаки «мёртвого» набора: тесты падают при каждом рефакторинге · новый баг с прода не ловится ни одним тестом · никто не помнит, что проверяет половина тестов.

Вопрос лекции: как отличить полезный тест от бесполезного — не на глазок, а системно?

Четыре столпа хорошего теста

Хориков, гл. 4: **Ценность теста = пойманные баги − ложные срабатывания − стоимость поддержки.**



Столпы 1 и 2 в конфликте: чем больше деталей проверяешь — тем лучше защита, но тем чаще тест падает при безобидных изменениях.

Столпы 2 и 4 не измеряются метрикой — их видит только человек при чтении кода. Отсюда — роль code review.

МОДУЛЬ 4 · ЛЕКЦИЯ 8

Метрики: покрытие и его пределы

Statement vs Branch Coverage

Line/Statement coverage — какие строки выполнились при прогоне.

Branch coverage — пройдены ли обе ветки каждого условия: и true, и false.

Классический пример Хорикова (гл. 1, листинг 1.1):

```
boolean isStringLong(String input) {
    if (input.length() > 5)    // ветка true — не покрыта
        return true;
    return false;              // покрыта тестом
}

@Test
void test() {
    boolean result = isStringLong("abc");  // длина 3 ≤ 5
    assertFalse(result);
}
```

Один зелёный тест, две метрики: строки — 4 из 5 (80%), ветки — 1 из 2 (50%).

Тот же метод после рефакторинга — тест не менялся:

```
boolean isStringLong(String input) {
    return input.length() > 5;  // все строки выполняются
}

@Test
void test() {
    boolean result = isStringLong("abc");  // тот же тест
    assertFalse(result);
}
```

Строки — 100%, ветки — те же 50%: line coverage обманывается компактной записью.

Правило СМП: каждый `if` в авторизации — пара тестов на обе ветки + граничные значения (модуль 2). Даже `if` без `else` имеет две ветки.

JaCoCo: отчёт и Quality Gate

Инструментирует байт-код (`prepare-agent` + `report`)

HTML — для людей: **жёлтый ромбик** = частично пройденное ветвление

XML — для машин: читает Quality Gate в CI

```
<execution>
  <id>check</id> <goals><goal>check</goal></goals>
  <configuration>
    <rules>
      <rule>
        <element>BUNDLE</element>
        <limits><limit>
          <counter>BRANCH</counter>
          <value>COVEREDRATIO</value>
          <minimum>0.70</minimum>
        </limit></limits>
      </rule>
    </rules>
  </configuration>
</execution>
```

Quality Gate: сборка падает, если branch coverage < 70%. Решение о мерже принимает машина, а не человек под давлением дедлайна.

Порог — договорённость команды: 70% по ветвям — достаточно для decline-сценариев, но не так много, чтобы тестировать геттеры.

Закон Гудхарта: пределы покрытия

«Когда метрика становится целью, она перестаёт быть хорошей метрикой»

Хороший негативный индикатор: 10% покрытия — однозначно плохо: 90% кода никто никогда не запускал в тестах.

Плохой позитивный индикатор: 100% — ничего не гарантирует: код запускался, но его результаты могли не проверяться.

Как обманывают метрику: тест без assert выполняет код — покрытие растёт. Рефакторинг `if` в тернарник — строк меньше, покрытие «улучшилось» само.

Как пользоваться: покрытие — гигиенический минимум и инструмент поиска дыр, а не цель. Смотрим на конкретные непокрытые ветки.

Мутационное тестирование (PITest)

Вопрос: как проверить, что тесты реально проверяют результат, а не просто выполняют код?

1. Мутация

PITest вносит микро-баги в байт-код: `>` → `⩾`, `+` → `-`, удаляет вызовы, `true` → `false`.

2. Тесты упали → мутация УБИТА

Баг обнаружен — тесты работают как надо.

2. Тесты зелёные → мутация ВЫЖИЛА

Изменение поведения никто не заметил — конкретная дыра в тестах.

Mutation Score — доля убитых мутаций. Честнее покрытия: обмануть приёмом «вызвать без assert» невозможно.

Цена: сотни мутантов × полный прогон. Запускают ночами, по расписанию, на критичной бизнес-логике.

Метафора: покрытие показывает, что пожарная сигнализация *установлена* в каждой комнате. Мутационное тестирование *поджигает спичку* и проверяет, что сигнализация сработает.

Интерактив: параметризованные тесты — выбери источник данных

[ИНТЕРАКТИВ — 5 мин] Четыре сценария. Для каждого хором называем источник данных и объясняем почему: `@ValueSource` · `@CsvSource` · `@EnumSource` · `@MethodSource`.

Сценарий 1: шесть невалидных PAN (короткий, буквы, неверная контрольная сумма)

Сценарий 2: авторизация — сумма × статус карты → решение, 8 комбинаций

Сценарий 3: отказ для ЛЮБОГО неактивного статуса карты

Сценарий 4: запросы с десятью полями, собранные билдером

```
// Разбор сценария 3 — новинка дня
@ParameterizedTest
@EnumSource(value = CardStatus.class, names = "ACTIVE",
            mode = EnumSource.Mode.EXCLUDE)
void shouldDeclineWhenCardNotActive(CardStatus status) {
    Decision decision = service.authorize(requestWith(cardWith(status)));
    assertThat(decision).isNotEqualTo(Decision.APPROVED);
}
```

Правило выбора: одно значение — `@ValueSource` · таблица примитивов — `@CsvSource` · перечисление — `@EnumSource` · сложные объекты — `@MethodSource`. И помним: шаблон `name` в отчёте, никаких if в теле теста.

МОДУЛЬ 4 · ЛЕКЦИЯ 8

Качество: читаемость, хрупкость, overmocking

Читаемость: тест как документация

Имя — оглавление поведения: `shouldDeclineTransactionWhenCardIsBlocked` читается как требование (лекция 7). Тест не вызывается из кода — длинное имя ничего не стоит.

Числа рассказывают историю: `assertEquals(1500, ...)` — откуда полторы тысячи? Лучше: `INITIAL_BALANCE - TRANSACTION_AMOUNT`.

Шум — в фабричные методы: `blockedCard()`, `cardWithLimit(5000)` — имя сценария прямо в месте использования. Не наследование, а приватные хелперы класса.

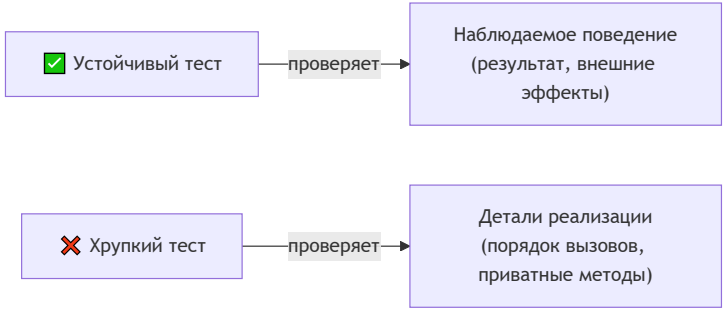
Mystery Guest (Месарош): тест зависит от данных, которых в нём не видно, — читатель прыгает по файлам, чтобы понять сценарий.

DRY в тестах — с осторожностью, правило RULE: Reduce, Reuse, but don't Over-abstract. Дублируйте то, что читателю важно видеть (входы и ожидания), абстрагируйте шум (инфраструктура).

Предостережение про @BeforeEach: setup нужен каждому тесту класса — иначе «пыльный» setup, переплетённый с телами. Разный контекст у групп тестов → `@Nested`-классы.

Хрупкие тесты: поведение vs реализация

Brittle test падает при изменении реализации, а не поведения. Причина почти всегда одна — **over-specification**: тест прибит к внутреннему устройству метода.



Цена ложных срабатываний: красный тест перестаёт быть сигналом → его игнорируют → отключают → набор мёртв.

Приватный метод «просит тест»? Это сигнал дизайна: логика просится в отдельный класс с публичным интерфейсом.

Критерий: меняется реализация — тест зелёный; меняется поведение — тест красный. Всё остальное — дефект теста, а не кода.

МОДУЛЬ 4 · ЛЕКЦИЯ 8

Overmocking и три стиля тестирования

Overmocking — моки там, где можно реальный объект. Симптомы: мок на каждую зависимость · verify больше, чем arrange · тест дублирует код тестируемого метода на языке Mockito.

Output-based

Проверяем возвращаемое значение. Функциональный стиль, самый устойчивый к рефакторингу.

State-based

Проверяем состояние объекта или системы после вызова. Всё ещё видно снаружи.

Communication-based

Проверяем взаимодействие через моки (verify). Самый хрупкий: прибит к деталям взаимодействия.

Порядок предпочтения: output > state > communication. Communication нужен для внешних эффектов: «уведомление отправлено», «запись в историю выполнена».

Вопрос для ревью: «можно ли этот тест переписать в более устойчивом стиле?» Каждая замена убирает крючок для рефакторинга.

Что тестировать: четыре типа кода

Два измерения: **сложность логики** × **количество зависимостей**. Отвечает на вопросы «надо ли тестировать контроллер? а DTO? а конфиг?»

Четыре типа кода (Хориков)

	Мало зависимостей	Много зависимостей
Сложная логика	**Доменная модель:** unit-тесты	**Переусложнённый код:** рефакторить
Простая логика	**Тривиальный код:** не тестируем	**Контроллеры:** интеграционные тесты

Доменная модель — золотая жила: расчёт лимитов, валидация транзакции. Максимальная отдача на вложенный час.

Humble Object: из переусложнённого кода выносим логику в чистый доменный класс без зависимостей; склейку проверяют интеграционные тесты (модуль 6).

Тестируемость — свойство дизайна. Тяжело тестируется unit-тестами → проблема в коде, а не в тестах.

Антипаттерны: знать врага в лицо

🍦 Ice Cream Cone
Много E2E, мало unit — перевёрнутая пирамида (модуль 3). Прогон часами, нестабильность.

🍽️ Generous Leftovers
Тест оставляет данные, следующие на них полагаются. Убрал первый — упал второй. Нарушение I из FIRST.

🐘 One Giant Test
Один метод проверяет пять вещей: создал-проверил-изменил-проверил-удалил. Не диагностирует при падении.

□ Leaking Domain Knowledge
Формула продублирована в тесте — вечно зелёный при общем баге. Или магическое ожидание без источника.

🤷 Test Logic in Production
`if (testMode)` в основном коде: мёртвый код в проде и скрытые ветки. Тестируемость — через DI.

Чек-лист ревьюера: перевёрнутая пирамида · щедрые остатки · гигантский тест · магические ожидания · тестовые ветки в проде. Любой из них — сигнал, что набор скоро начнёт болеть.

Интерактив: найди дефекты

[ИНТЕРАКТИВ — 5 мин] Четыре теста из реальных пулл-реквестов (упрощённо). Найдите дефект и назовите нарушенный принцип.

```
// Тест 1
@Test
void processTransaction() {
    service.process(validRequest());
}

// Тест 2
@Test
void shouldAuthorize() {
    when(repository.findCard(PAN))
        .thenReturn(activeCard());
    AuthorizeResult result = service.authorize(request);
    verify(repository).findCard(PAN);    // ← ?
    assertEquals(AuthorizeResult.APPROVED, result);
}
```

```
// Тест 3
@Test
void shouldFailOnBlockedCard() {
    assertThrows(Exception.class,
        () → service.authorize(blockedRequest()));
}

// Тесты 4-8: пять методов
// shouldDecline1()...shouldDecline5(),
// отличающихся только суммой и статусом карты
```

МОДУЛЬ 4 · ЛЕКЦИЯ 8

Code Review тестов: чек-лист ревьюера

Тесты — production-код: проходят ревью наравне с основным. Два из четырёх столпов не измеряются метрикой — их видит только человек.

1. Имя описывает сценарий и ожидание?

2. Проверяется поведение, а не реализация?

3. Есть ли негативные сценарии, не только happy path?

4. Моки по делу: SUT не замокирован, verify — только команды?

5. Нет ли магических чисел и Mystery Guest?

Стоп-слова ревьюера: `test1` · `verify(repository, times(3))` · `@Mock` на тестируемом классе · `if` в теле теста.

Порядок чтения: имена → тела → моки → данные. Предлагайте исправление, а не только замечание.

Итоги модуля



Четыре столпа — рамка для любого теста

Защита от регрессий · устойчивость к рефакторингу · быстрая обратная связь · поддерживаемость. Спорите о тесте — спросите, какой столп шатается.



Покрытие — инструмент поиска дыр

Хороший негативный, плохой позитивный индикатор. Смотрим на branch, а не на line. Мутации проверяют, что тесты ловят баги.



Качество теста видит только ревьюер

Устойчивость и поддерживаемость не измеряются метриками. Code review — единственный контрольно-пропускной пункт.



AAA и FIRST — гигиена каждого теста

Unit-тесты — основание пирамиды: бизнес-логика, быстро, изолированно. Баг, пойманный unit-тестом, стоит минуты; баг с прода — инцидент.

Дальше — модуль 5: тестирование API. Те же FIRST и AAA, но вместо моков — реальные сервисы в Testcontainers.

Источники

Khorikov, V. (2020). *Unit Testing: Principles, Practices, and Patterns*. Manning. — Гл. 1 (цели и метрики), гл. 3 (структура и именование), гл. 4 (четыре столпа), гл. 6 (покрытие и мутационное тестирование), гл. 8 (четыре типа кода, Humble Object), гл. 11 (антипаттерны).

Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley. — Mystery Guest, Generous Leftovers, антипаттерны тестов.

Freeman, S., & Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley. — Тесты как живая документация.

Beck, K. (2002). *Test Driven Development: By Example*. Addison-Wesley. — Изолированные тесты (гл. 25), «страх → скука».

Crispin, L., & Gregory, J. (2009). *Agile Testing*. Addison-Wesley. — Пирамида тестирования.

Tudose, C. (2020). *JUnit in Action* (3rd ed.). Manning. — JUnit 5: аннотации, параметризация.

Coles, H. PIT Mutation Testing. pitest.org — документация PITest, мутационные операторы.

JaCoCo Java Code Coverage Library. jacoco.org — счётчики line/branch, цель check.


Вопросы?